NED University of Engineering and Technology

Computer Science and Information Technology

COMPLEX COMPUTING PROBLEM (CCP)
PROJECT SUBMISSION

Artificial Intelligence & Expert System (CT-361)

AI-Based Source-Code Vulnerability Detection



Student Details

Sr. No.	Student Name	Roll No.
1	Sana Zehra	CT-24062
2	Aleeza Mujahid	CT-24065
3	Fatima Hasan	CT-24064

Abstract

This project investigates automated binary classification of potentially vulnerable C/C++ source-code functions. The workflow integrates the BigVul and CVEfixes datasets, audits and cleans the records, constructs leakage-aware commit-grouped BigVul splits, examines class imbalance and project bias, trains TF-IDF/Logistic Regression baselines, and fine-tunes CodeBERT. The resulting models are evaluated on held-out BigVul data and an independent CVEfixes dataset. Although fine-tuning improves CodeBERT over its initial state, the final test F1 is 0.223 and external transfer remains close to chance. A key contribution is detecting and mitigating a misleading whitespace artifact that initially inflated validation results.

1. Project Overview and Research Objectives

The system accepts a C/C++ function and produces a model-based vulnerability score and binary flag. Label 1 indicates a dataset-labeled vulnerable or pre-fix function; label 0 indicates a dataset-labeled non-vulnerable or post-fix function. Neither label guarantees the absence or presence of an exploitable flaw in every case. Build a reproducible data pipeline from real vulnerability datasets. Prevent leakage from related code and fixing commits. Study class imbalance, code length, and project concentration. Compare a trivial majority baseline, TF-IDF/Logistic Regression, and fine-tuned CodeBERT. Evaluate using accuracy, precision, recall, F1, PR-AUC, and ROC-AUC. Test out-of-dataset generalization and disclose shortcomings. Provide a safe inference interface that does not execute submitted source code.

2. Research Papers and Dataset Sources

BigVul

BigVul originates from the research paper "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries" by Fan et al. (2020). The authors mined public CVE information and related GitHub repositories to connect vulnerability descriptions, code changes, and C/C++ code. The original publication reports 3,754 vulnerabilities across 91 weakness types from 348 projects; these are publication-level figures, not the row counts of the specific function-level export used here.CVEfixes

CVEfixes

originates from the research paper "CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software" by Bhandari, Naseer and Moonen (2021). It automates collecting CVE-linked repositories and fixes and stores linked CVE, commit, file, and method-level information in a relational database. The project used a v1.0.8 export of C/C++ functions; the original paper describes an earlier release, so paper-wide counts must not be confused with this export.

Related model and methodology literature

CodeBERT: Feng et al. (2020), "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," describes pretrained representations of source code and natural language. PrimeVul and SIVA documents are present in the project repository as research references; the available evidence does not establish that their datasets were used for training. The two documented data sources are BigVul and CVEfixes.

3. Dataset Collection and Preprocessing

Dataset	Raw records	Role
BigVul	188,636	Primary training/validation/test source
CVEfixes v1.0.8 C/C++ export	20,060	Independent external evaluation source
Combined raw total	208,696	Input to standardization and cleaning

Data is tabular (CSV/ZIP and exported database tables) with unstructured C/C++ source code stored in string fields and structured metadata such as labels, project names, commit hashes, CVE/CWE identifiers, and dates. Large data processing was supported by Python/Colab-style workflows, selective columns, chunked reading, and compressed CSV outputs. The project does not train on CVE identifiers as shortcut features.

Cleaning and deduplication

The cleaning pipeline harmonizes schemas, validates code and binary labels, restricts languages to C/C++, removes code shorter than 20 characters, hashes code after collapsing whitespace, removes conflicting labels and exact normalized duplicates, removes overlap between BigVul and CVEfixes, and standardizes commit identifiers and dates. An audit report records removals. MD5 is used as a compact deduplication fingerprint, not as a security primitive.

Stage	BigVul	CVEfixes
Raw	188,636	20,060
Short code removed	47	7
Conflicting-label rows removed	2,538	1,209
Duplicate rows removed	1,251	345
Cross-dataset code overlap removed	—	2,764
Cleaned	184,800	15,735
CVEfixes commit overlap excluded at split stage	—	3,826
Final external evaluation	—	11,909

Cleaned total = 200,535 functions (184,800 + 15,735). The split-ready total is 196,709 after excluding 3,826 CVEfixes records with overlapping commits. Source: project cleaning audit and split report.

4. Data Splitting and Exploratory Data Analysis

BigVul was split 70/15/15 into train (129,360), validation (27,720), and test (27,720), using seed 42. Rather than assigning individual rows randomly, a greedy algorithm grouped functions by Git commit and assigned complete groups while approximating target sizes and the vulnerable class fraction. Automated checks guarded against shared commits, sample IDs, and normalized code hashes across splits. This is not equivalent to a project-disjoint or time-based evaluation.

Split	Rows	Vulnerable	Non-vulnerable	Vulnerable %
Train	129,360	6,748	122,612	5.2%
Validation	27,720	1,446	26,274	5.2%
Test	27,720	1,446	26,274	5.2%
CVEfixes external	11,909	4,754	7,155	39.9%

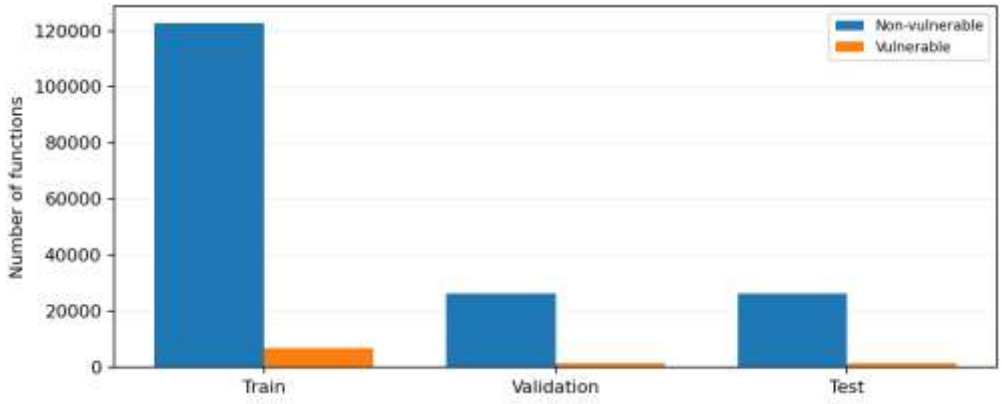


Figure 1. Class distribution across splits.

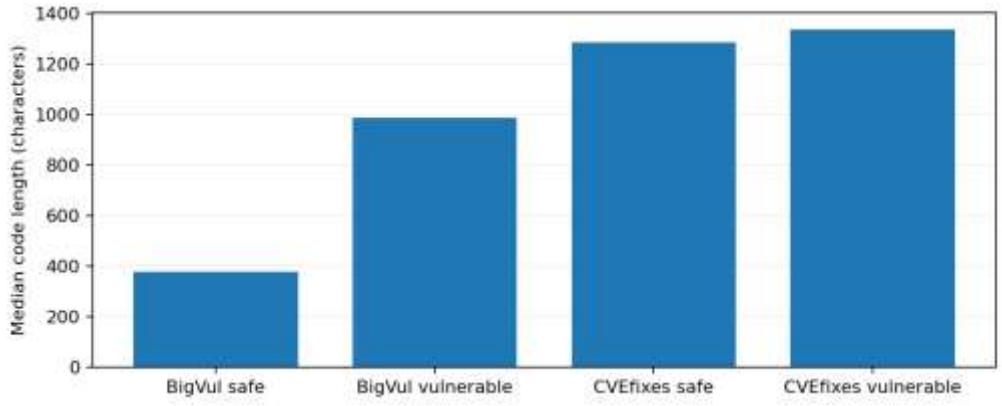


Figure 2. Median code length by label.

BigVul comprises 182,167 C and 2,633 C++ functions. Its top two projects account for roughly 66% of rows. Vulnerable BigVul functions are longer at the median (988 versus 377 characters). These differences can introduce project-identity and function-length shortcuts. EDA was conducted after the splits were frozen and did not alter them.

5. CVEfixes Label Validation

CVEfixes before/after records are useful but are not perfect vulnerability labels: a fixing commit can modify functions that were not themselves exploitable. The project matched before/after function versions and constructed a stricter CVEfixes subset of 5,266 rows (2,628 pre-fix and 2,638 post-fix) from 1,499 commits and 470 projects. This subset supports a more controlled external check, while remaining subject to label noise.

6. Machine Learning and Deep Learning Models

6.1 Majority-class baseline

Predicts label 0 for every function. On BigVul its accuracy is 0.948 but its recall, precision, and F1 are all zero. It establishes why accuracy is a poor standalone measure with 5.2% positive labels.

6.2 TF-IDF and Logistic Regression

The source-code text is tokenized into unigram and bigram features. TF-IDF weights informative tokens; a regularized Logistic Regression classifier maps the sparse features to a score. The baseline uses class_weight="balanced" to increase the loss contribution of rare positive samples and chooses its threshold using validation F1. The regularization parameter C was searched over 0.1, 1.0, and 10.0; C=1.0 was selected by validation PR-AUC.

TF-IDF(t,d) = TF(t,d) x log(N / DF(t)) (conceptual form; implementations may smooth or

normalize)

Logistic score: sigmoid(w^T x + b) = 1 / (1 + exp(-(w^T x + b)))

Balanced class weight: w_c = N / (K x N_c)

6.3 Generic-token baseline

To reduce dependence on project-specific identifiers, the generic-token preprocessing retains identifiers found in at least five distinct training projects and replaces rarer identifiers with ID. TF-IDF and Logistic Regression then operate on the transformed tokens. This approach trades some lexical detail for a potential reduction in project-identity shortcuts. The final saved generic baseline uses threshold 0.637.

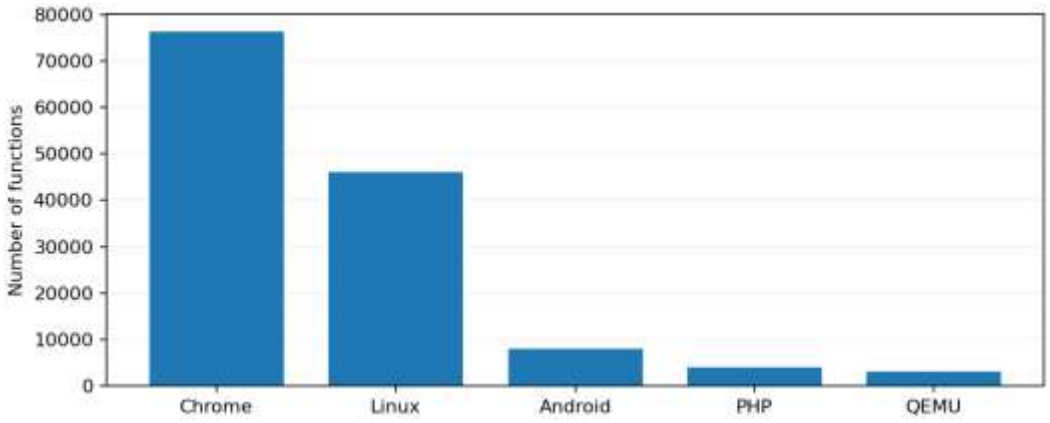


Figure 3. Largest BigVul source projects.

6.4 CodeBERT fine-tuning

CodeBERT (microsoft/codebert-base) was initialized with pretrained weights and a new two-class classification head. The project trained on a subsample of 46,748 BigVul training functions (6,748 vulnerable plus 40,000 non-vulnerable), using class weights [0.584, 3.464], learning rate 2e-5, batch size 16, two epochs, seed 42, and maximum sequence length 512. Code text was normalized by stripping per-line whitespace, collapsing spaces/tabs, and removing blank lines. The best validation epoch was epoch 2 and the final frozen score threshold was 0.66.

Weighted cross-entropy: L = - sum_c w_c y_c log(p_c) (per example)

A 512-token maximum means long functions are truncated; the application must warn users when this happens. The model score should not be described as a calibrated probability.

7. Performance Metrics and Mathematical Formulas

TP = correctly flagged vulnerable; TN = correctly flagged non-vulnerable; FP = non-vulnerable flagged; FN = vulnerable

missed.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall (sensitivity) = TP / (TP + FN)

F1 = 2 x Precision x Recall / (Precision + Recall)

ROC-AUC = area under the true-positive-rate vs false-positive-rate curve

PR-AUC = area under the precision-recall curve across score thresholds

With a rare positive class, precision, recall, F1 and PR-AUC are especially informative. A threshold changes the classification trade-off, but not the ranking-based ROC-AUC or PR-AUC for fixed scores.

8. Experimental Results and Before-and-After Comparisons

8.1 Original baseline validation (not final test)

Model / threshold	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Majority / 0.5	.948	.000	.000	.000	.052	.500
TF-IDF / 0.5	.845	.166	.492	.248	.188	.765
TF-IDF / 0.636	.903	.224	.351	.273	.188	.765

At the chosen threshold, the baseline validation confusion matrix was TN=24,511, FP=1,763, FN=938, TP=508.

8.2 CodeBERT validation before/after fine-tuning

Stage	Accuracy*	Precision*	Recall*	F1*	PR-AUC	ROC-AUC
Epoch 0	.127	.053	.938	.101	.046	.476
Epoch 1	.871	.172	.387	.239	.148	.759
Epoch 2	.877	.181	.385	.246	.156	.768

*Accuracy, precision, recall, and F1 in this table use each stage's best-F1 validation threshold; epoch-0 at fixed threshold 0.5 instead had accuracy .948 and zero precision, recall, and F1. These are validation figures, not test results.

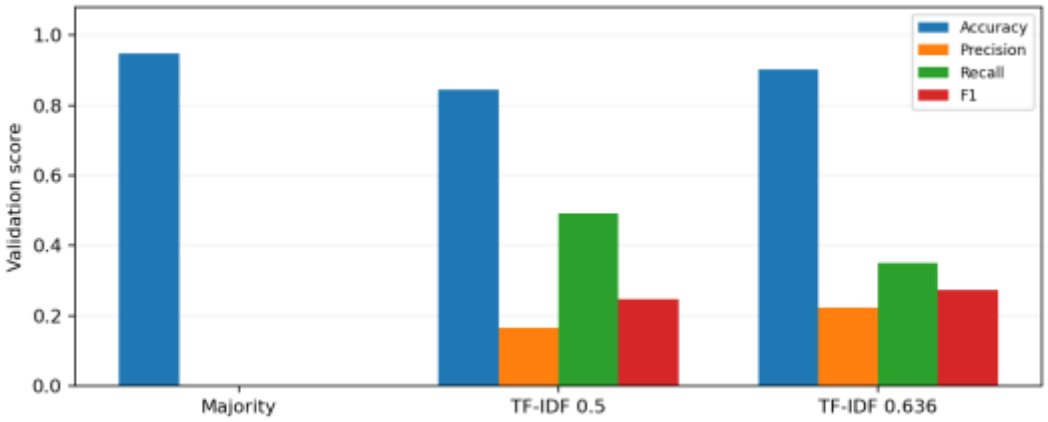


Figure 4. Original baseline validation comparison.

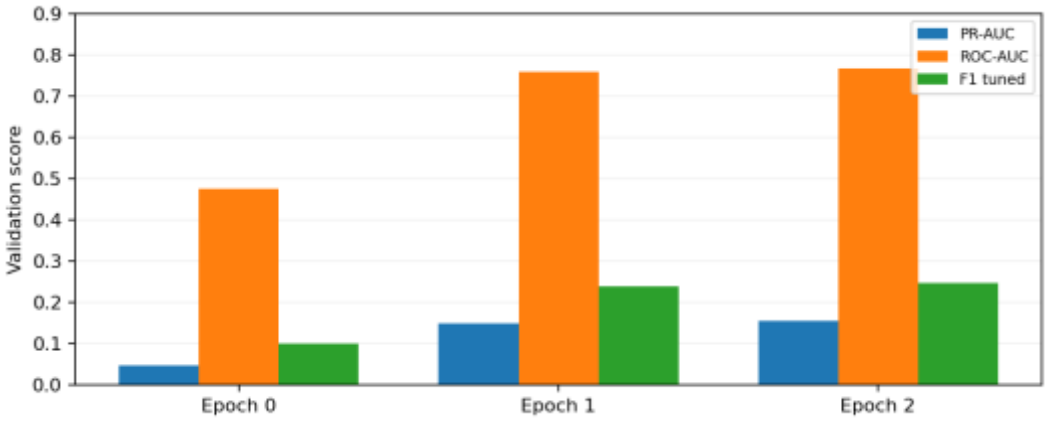


Figure 5. CodeBERT validation before and after fine-tuning.

8.3 Final held-out BigVul test (27,720 functions)

Model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Majority	.948	.000	.000	.000	.052	.500
Generic TF-IDF + LR	.882	.163	.305	.212	.159	.720
CodeBERT before	.948	.000	.000	.000	.045	.466
CodeBERT after	.881	.169	.327	.223	.151	.740

Model	TN	FP	FN	TP
Generic TF-IDF	24,006	2,268	1,005	441
CodeBERT after	23,954	2,320	973	473

The reported bootstrap difference in PR-AUC (CodeBERT minus TF-IDF) is -0.007 with 95% interval [-0.019, +0.006]; the interval includes zero, so superiority of CodeBERT is not established. A higher test F1 alone should not be overstated.

8.4 External CVEfixes evaluation

Set / model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Full / Generic TF-IDF	.523	.398	.379	.389	.401	.506
Full / CodeBERT	.526	.396	.358	.376	.392	.501
Strict / Generic TF-IDF	.491	.488	.422	.453	.493	.490
Strict / CodeBERT	.497	.495	.401	.443	.494	.490

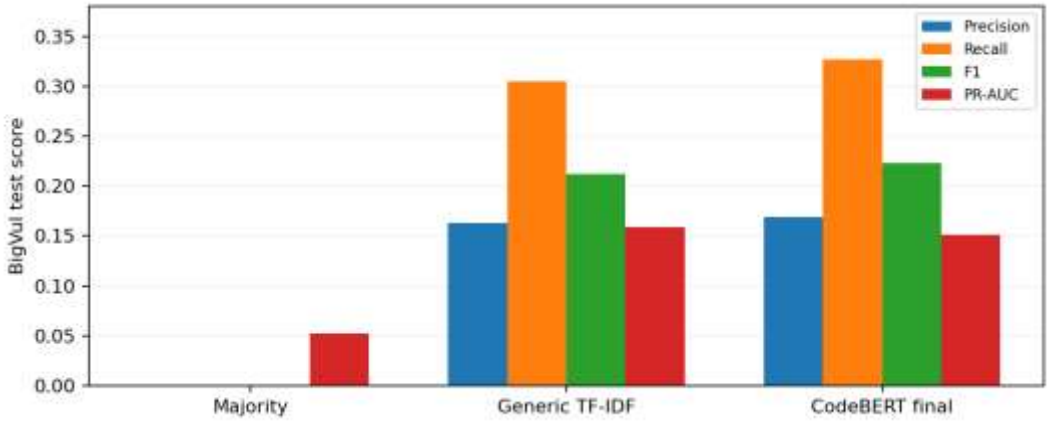


Figure 6. Held-out BigVul test model comparison.

For the strict before/after matched pairs, CodeBERT assigned a higher score to the pre-fix version in only 33.6% of 2,548 pairs; generic TF-IDF did so in 34.5%. This suggests serious limitations in transferring the learned patterns to the external data.

9. Data Artifacts and Corrective Experiments

A major finding was an association between trailing whitespace and the BigVul labels: 56.1% of vulnerable training functions versus 1.9% of non-vulnerable functions contained a line with trailing whitespace. An initial CodeBERT experiment on raw code achieved validation PR-AUC 0.864 and F1 about 0.81, but formatting-only features achieved PR-AUC 0.632, revealing a strong spurious cue. After code normalization, the final CodeBERT results were substantially lower but more credible. The TF-IDF baseline largely ignored whitespace and was not affected in the same way. This early raw-code experiment must not be presented as the final model performance. Normalization must be identical during training and inference.

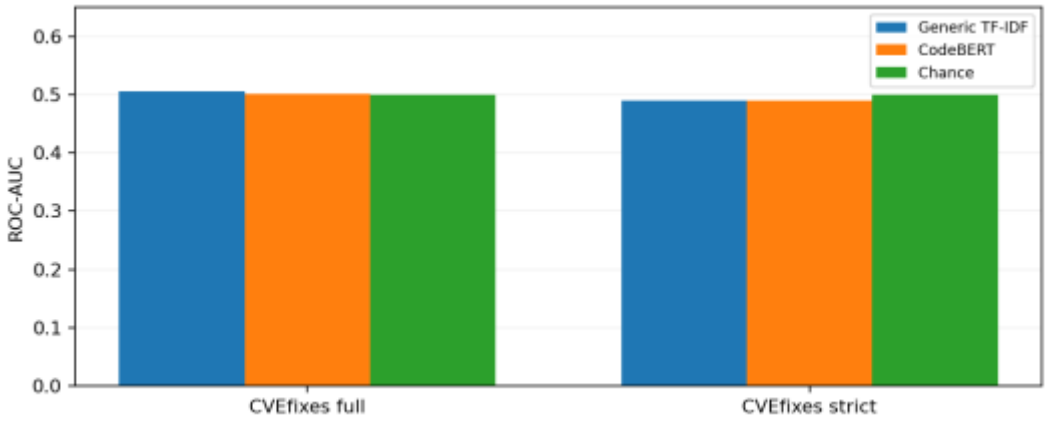


Figure 7. External ROC-AUC comparison.

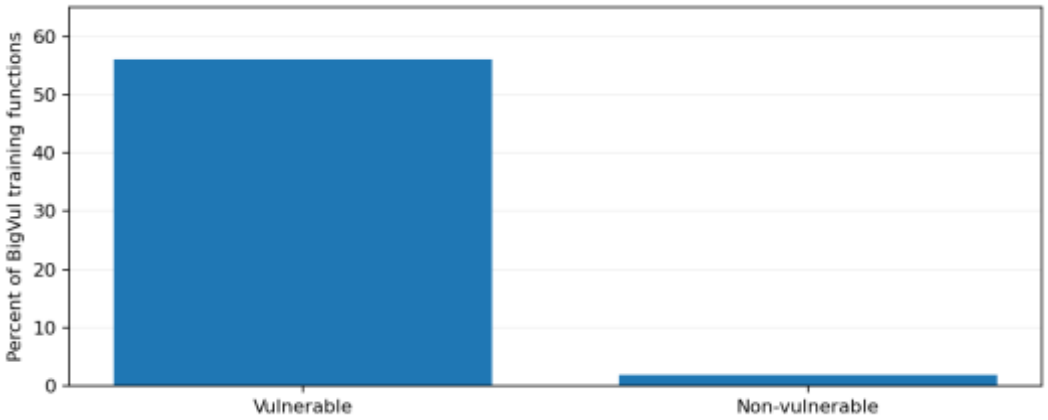


Figure 8. Whitespace shortcut artifact.

10. Application and Prediction Workflow

The repository contains an app folder, run.py, tests, and an inference script. The documented handoff specifies the following safe inference contract: (1) accept C/C++ source text, (2) apply the same normalize_code function used in training, (3) tokenize with a maximum of 512 tokens, (4) load CodeBERT weights once, (5) produce a model score, (6) flag as "potentially vulnerable" if score >= 0.66, otherwise "not flagged", and (7) warn on truncation. The submitted code must never be executed and must be escaped when displayed. The detailed app UI and framework behavior were not independently verified for this report.

11. Limitations and Future Improvements

Severe class imbalance in BigVul (5.2% positives) and changing class prevalence on CVEfixes. Noisy labels: pre-fix does not always mean a function is demonstrably vulnerable; post-fix does not prove security. Project concentration: Chrome and Linux dominate BigVul; commit-disjoint is not project-disjoint. Shortcut artifacts: trailing whitespace and code length can predict labels without indicating vulnerability. CodeBERT truncation: 13.0% of normalized BigVul test functions exceed 512 tokens, disproportionately affecting vulnerable functions. Limited deep-model exploration: one run, two epochs, a 46,748-row training subsample, and limited hyperparameter search. Weak external generalization: ROC-AUC near 0.50 on CVEfixes, including the strict subset. Model scores are not calibrated probabilities; a negative flag is not a security guarantee. Near-duplicate assessment was limited; stronger semantic deduplication, project/time splits, and independent manual label review would help. Future work should include better function-level labels, project-disjoint and temporal evaluations, longer-context models, repeated seeds and confidence intervals, robust normalization and shortcut audits, calibration analysis, and broader vulnerability-category analysis.

12. Conclusion

The project establishes a reproducible end-to-end study of source-code vulnerability detection using real-world C/C++ datasets. It delivers a documented cleaning and leakage-aware splitting pipeline, quantitative EDA, interpretable classical baselines, fine-tuned CodeBERT, and internal and external evaluations. CodeBERT improved over its initial state on BigVul, but neither CodeBERT nor the generic TF-IDF baseline generalized reliably to CVEfixes. The strongest scientific contribution is the transparent investigation of data leakage, label noise, class imbalance, and misleading shortcut features. The current detector should be treated as a research prototype rather than a dependable security scanner.

The complete project source code and supporting reports are available at https://github.com/AleezaMujahid/AIES_PROJECT